Architecturally, the GWT is designed around four main components that cover almost all the operations that it is capable of undertaking:

1. **GWT Java-to-JavaScript Compiler:** this translates the code written in Java to JavaScript.
2. **GWT Hosted Web Browser:** the browser is for running and executing the GWT applications in hosted mode. When this happens, the code runs as Java in the JVM without compiling to JavaScript.

3. **JRE emulation library:** these are the JavaScript implementations of the most widely-used classes in the Java standard class library. For example, the java.lang package classes.

4. **GWT Web UI class library:** the library is a set of custom interfaces and classes for creating widgets—like buttons, images, and text. This is the core user interface library used to create GWT applications.

3.5 How To Use GWT

1. To begin, programmers need to prepare a program (client-side code) using any Java IDE. For writing the code, you have access to core Java classes and a library of classes (included in the GWT) that provide features very similar to that of JavaScript. For example, the Document Object Model, alert boxes, etc.

Then there are Java classes for adding widgets to the page, from simple buttons to complex drop-down menus and trees. All these widgets are event-controlled, so one can prepare code to respond to them.

2. Once you're done writing and debugging your code and are ready to test application in a browser, all you need to do is to compile the file in the preferred Java IDE and launch the GWT

Shell. This will pop up a specialised browser window and load your application. On Windows, that browser window uses the Internet Explorer rendering engine, and Mozilla on Linux.


The GWT Shell

The GWT Shell is the interface between the Java classes and the special browser window, allowing the application's client-side logic to run within the browser even though it is implemented in Java, not JavaScript. It allows you to test and debug your client-side logic with ease. You can set breakpoints to pause and step through client-side event handlers, and write unit tests to verify that the user interface works as per the intended design.

3. Thus far, even though our code has been tested in a browser as if it were actual Web content, it is actually still a collection of Java classes, and the final step is to compile these classes to efficient cross-browser JavaScript code. The GWT compiler then reads the source code and generates the equivalent JavaScript code. Thereafter, the generated JavaScript code (along with the static HTML, CSS and image files) is ready to be uploaded to your server. The resulting code is entirely self-contained—it requires no browser plugins or special server technology.

3.6 Concluding Remarks

GWT is an extremely interesting and useful development environment for Java developers (especially) who want to produce rich Web clients and interactive Web content. Its strength lies in the fact that it provides a powerful develop-debug-deploy environment that can exploit the full program creation and debugging features in IDEs such as Eclipse.